

# Proposed Solutions for the DRI Website Redesign

Directia de Relatii Internationale  
West University of Timișoara

**Focus:** Information architecture, user journeys, content governance, and technical justification for a Headless WordPress + React implementation.

Technology Stack: Headless WordPress + React SPA  
Audit-Based Redesign Proposal

# Contents

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                | <b>2</b> |
| <b>2</b> | <b>Observed Problems</b>                           | <b>2</b> |
| <b>3</b> | <b>Proposed Solutions</b>                          | <b>2</b> |
| 3.1      | Rebuild the Information Architecture . . . . .     | 2        |
| 3.2      | Replace Document Walls with Guided Pages . . . . . | 3        |
| 3.3      | Create Clear User Journeys . . . . .               | 3        |
| 3.4      | Introduce Structured Content Types . . . . .       | 3        |
| 3.5      | Fix Broken Links and Legacy Routes . . . . .       | 4        |
| 3.6      | Improve SEO and Discoverability . . . . .          | 4        |
| <b>4</b> | <b>Headless WordPress + React</b>                  | <b>4</b> |
| 4.1      | Content Delivery Pipeline . . . . .                | 4        |
| 4.2      | Modern . . . . .                                   | 5        |
| 4.3      | Dependable . . . . .                               | 5        |
| 4.4      | Persistent . . . . .                               | 5        |
| 4.5      | How Secure Communication Will Be Ensured . . . . . | 6        |
| 4.5.1    | HTTPS / TLS Encryption . . . . .                   | 6        |
| 4.5.2    | Restricted REST API Exposure . . . . .             | 6        |
| 4.5.3    | Authentication for Protected Actions . . . . .     | 6        |
| 4.5.4    | CORS Policy Control . . . . .                      | 7        |
| 4.5.5    | Input Validation and Sanitisation . . . . .        | 7        |
| 4.5.6    | Server Hardening . . . . .                         | 7        |
| 4.5.7    | Practical Headless Security Benefit . . . . .      | 7        |
| <b>5</b> | <b>Conclusion</b>                                  | <b>8</b> |

## 1. Introduction

This document proposes concrete solutions for the problems observed in the audit of the current DRI website. The goal is to transform the existing website from a document-heavy WordPress information site into a modern, structured, user-oriented digital platform.

The redesign should not merely restyle the old site. It should clarify routes, reduce user confusion, guide visitors through clear institutional processes, and preserve WordPress as a dependable content-management layer.

## 2. Observed Problems

The audit shows several structural and usability problems:

- duplicated Erasmus route trees;
- broken links in important areas such as the homepage and footer;
- exposed test, example, and trashed pages;
- inconsistent Romanian and English URL structures;
- weak internal linking between related pages;
- heavy dependence on PDF, DOCX, and XLSX files;
- no clear news or announcements system;
- public pages affected by `noindex`, `nofollow`.

## 3. Proposed Solutions

### 3.1 Rebuild the Information Architecture

The new website should use a clean, audience-first structure:

**Recommended main areas:**

About DRI, Erasmus, International Students, Programmes, Partnerships, News, Contact.

Each section should contain clear subroutes instead of scattered legacy pages. For example:

```
/erasmus/incoming-students/  
/erasmus/outgoing-students/  
/international-students/non-eu/  
/international-students/eu/  
/partnerships/visiting-uvvt/
```

## 3.2 Replace Document Walls with Guided Pages

Many current user journeys end in downloadable documents. Instead, procedures should be transformed into structured HTML pages.

### Recommended page pattern:

Overview → Eligibility → Required Documents → Deadlines → Application Steps → Contact.

Files should remain available, but only as supporting downloads, not as the primary way of understanding the process.

## 3.3 Create Clear User Journeys

Different visitors have different needs. The redesign should guide them directly.

| User Type                | Recommended Journey                                                     |
|--------------------------|-------------------------------------------------------------------------|
| Incoming Erasmus Student | Nomination, application, accommodation, arrival, academic calendar      |
| Outgoing UVT Student     | Available places, application, selection, funding, final documents      |
| International Applicant  | Citizenship path, eligibility, required files, application platform     |
| Staff Member             | Teaching mobility, training mobility, current calls, required documents |
| Partner Institution      | Agreements, Visiting@UVT, contact points, collaboration process         |

## 3.4 Introduce Structured Content Types

The current site mixes procedures, stories, calls, documents, and old yearly offers. The redesign should separate them into content types:

- **Guides** – stable procedural pages;
- **Calls** – time-based opportunities;
- **Documents** – downloadable files;
- **Stories** – student or staff experiences;
- **Programmes** – institutional programme descriptions;
- **Announcements** – official news and updates.

### 3.5 Fix Broken Links and Legacy Routes

All broken links should be corrected before launch. Legacy routes should not simply disappear; they should redirect to the new canonical routes.

**Rule:** one content area should have one official route. Old URLs should redirect instead of competing with the new structure.

### 3.6 Improve SEO and Discoverability

The production site should remove global `noindex`, `nofollow`. Each important page should have:

- a clear title;
- a meta description;
- canonical URL;
- structured headings;
- internal links to related pages.

## 4. Headless WordPress + React

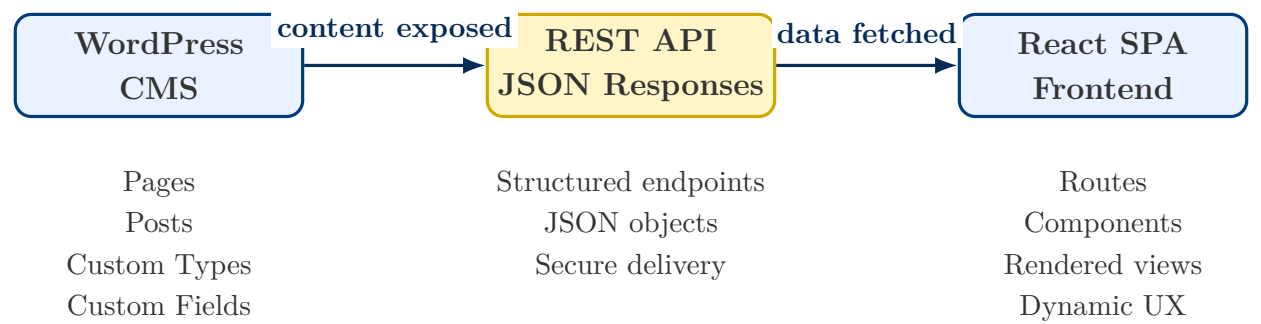
### 4.1 Content Delivery Pipeline

The platform uses a headless architecture where WordPress manages editorial content, the REST API exposes that content as structured JSON, and React renders the public-facing website.

In WordPress, administrators and staff members create:

- **Pages** – static institutional content such as About, Contact, or Partnerships;
- **Posts** – news, announcements, updates, and time-sensitive content;
- **Custom Post Types** – specialised entities such as Programmes, Opportunities, Staff Members, Partner Universities, or Documents;
- **Custom Fields** – structured metadata such as deadlines, coordinators, countries, links, PDF files, and categories.

These content types are then requested by the React frontend through the WordPress REST API in JSON format.



**Example flow:**  
A staff member publishes an Erasmus announcement or updates a Programme entry in WordPress. React then retrieves that data through endpoints such as:

```
/wp-json/wp/v2/posts  
/wp-json/wp/v2/pages  
/wp-json/wp/v2/programmes  
/wp-json/wp/v2/partners
```

The frontend transforms the received JSON into modern UI components, searchable lists, programme pages, and user-friendly navigation flows.

4.2 Modern

React allows the frontend to behave like a modern single-page application. This improves navigation, page transitions, filtering, reusable components, and guided user flows.

4.3 Dependable

WordPress remains the content management system. This means editors can still manage institutional content through a familiar interface, while developers can build a cleaner and more controlled frontend.

4.4 Persistent

The architecture is sustainable because content and interface are separated:

| Layer        | Responsibility                                                              |
|--------------|-----------------------------------------------------------------------------|
| WordPress    | Stores pages, documents, announcements, translations, and editorial content |
| REST API     | Delivers structured JSON content to the frontend                            |
| React SPA    | Renders the modern user interface and navigation experience                 |
| Vite         | Provides fast development and optimized production builds                   |
| Tailwind CSS | Enables consistent institutional styling                                    |

This approach keeps the institutional reliability of WordPress while allowing the frontend to become faster, clearer, and more maintainable.

## 4.5 How Secure Communication Will Be Ensured

Because the architecture separates backend and frontend systems, secure communication between WordPress and the React application must be intentionally configured. The project will apply multiple security layers.

**Security Principle:** Only authorised and encrypted data exchange should occur between WordPress and the frontend application.

### 4.5.1 HTTPS / TLS Encryption

All traffic between:

- visitors and the React frontend;
- React frontend and WordPress REST API;
- administrators and the WordPress dashboard;

will use HTTPS with valid TLS certificates. This prevents interception, tampering, and credential leakage.

### 4.5.2 Restricted REST API Exposure

Public endpoints will expose only information intended for visitors, such as:

- public pages;
- announcements;
- programme descriptions;
- approved downloadable resources.

Sensitive administrative data, drafts, private users, and internal metadata will remain inaccessible.

### 4.5.3 Authentication for Protected Actions

Any action involving editing, creation, deletion, or privileged access will require authentication through secure methods such as:

- WordPress session authentication (dashboard users);
- Application Passwords;
- JWT / token-based authentication (if custom services are needed);
- role-based permission checks.

#### 4.5.4 CORS Policy Control

Cross-Origin Resource Sharing (CORS) will be configured so that only approved frontend domains may request protected API resources.

Example: only the official React production domain and approved development environments may communicate with the API.

#### 4.5.5 Input Validation and Sanitisation

All submitted forms (contact forms, applications, uploads, search parameters) must be validated and sanitised to prevent:

- SQL injection;
- cross-site scripting (XSS);
- malicious uploads;
- spam abuse.

#### 4.5.6 Server Hardening

The WordPress server should additionally implement:

- firewall rules;
- rate limiting;
- automatic updates;
- plugin minimisation;
- backup routines;
- brute-force login protection.

#### 4.5.7 Practical Headless Security Benefit

Because visitors do not directly use the WordPress theme layer, the public attack surface is reduced. Users interact mainly with the React frontend, while WordPress remains the managed content backend.

This architecture improves security when properly maintained, because the CMS administration area is separated from the public user interface.



## 5. Conclusion

The redesign should solve the old site's structural problems, not only improve its appearance. The proposed solution introduces a cleaner information architecture, guided user journeys, structured content types, stronger SEO, and a modern technical foundation.

Headless WordPress + React is appropriate because it combines institutional stability with modern frontend flexibility.